Telegram 3-Channel Movie Automation Bot

This project is an admin-only Telegram bot built with **Python 3.11+** and `python-telegram-bot` v20+. It implements a controlled workflow for copying an accessible Telegram video into `@nightflixdrive`, fetching movie metadata from OMDb, generating Burmese promotional copy with OpenAI, collecting a poster, showing a preview, and publishing to the Club channel before the Main channel.

> **Important Telegram limitation:** The bot can copy a message only when Telegram allows the bot to access the source chat and message. Add the bot to the source channel when required, and do not use this project to bypass protected content, private-channel access controls, or copyright restrictions. The source link should be a public username link such as `https://t.me/source_channel/123`.

Files

File	Purpose
`bot.py`	Complete async bot application and conversation workflow
`requirements.txt`	Python dependencies
`Procfile`	Render web-service start command
`.env.example`	Required and optional environment variables
`README.md`	Setup and deployment instructions

Workflow

The administrator sends `/start` or `/new`, then sends the external Telegram message link and movie name. The bot uses `copy_message` to place the source message in the Drive channel and creates the new Drive link from the copied message ID. It then requests OMDb metadata and asks OpenAI for a short Burmese introduction and a detailed Burmese review.

The administrator uploads a poster photo. The bot displays the Club and Main content in a final preview with **Approve & Post All** and **Cancel** buttons. Approval publishes the poster and Club review first, derives the Club post link, and then publishes the Main post containing both links. If Club publication fails, Main publication is not attempted.

Telegram setup

Create the bot through BotFather and copy its token. Add the bot as an administrator to `@nightflixdrive`, `@nightflixclub`, and `@nightflixmyanmar`. Give it permission to post messages, post photos, and copy messages where the Telegram UI exposes that permission.

The bot does not need to be an administrator in every external source channel, but it must be able to access the linked message. For channels that restrict access, add the bot to that source channel or provide a source that Telegram permits the bot to read. The target channels should have public usernames if you want links in the requested `https://t.me/channel/message_id` format.

Find an administrator's numeric Telegram user ID using a trusted ID bot or a controlled development message, then put that number in `ADMIN_IDS`. Do not use a username for authorization because Telegram usernames can change.

API setup

Create an OMDb API key at omdbapi.com. Create an OpenAI API key from the OpenAI platform. The default model is `gpt-4o-mini`; set `OPENAI_MODEL` to another compatible chat-completions model if your account or endpoint requires it. The bot uses structured JSON output so the Burmese short intro and detailed review remain separate and predictable.

Copy `.env.example` to `.env` for local development and replace every placeholder. Never commit `.env` or expose either API key in Telegram messages.

Local development

Bash

```
cd /home/ubuntu/telegram-movie-bot
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Edit .env, then run:
python bot.py
```

When `RENDER_EXTERNAL_URL` is absent, the application uses polling. Press `Ctrl+C` to stop it. Only one polling process may be connected to a bot token at a time.

Render deployment

Create a new **Web Service** on Render, connect the repository containing this directory, and set the root directory to `telegram-movie-bot` if the repository contains multiple projects. Render detects the `Procfile` and runs `python bot.py`.

Add all variables from `.env.example` in Render's Environment settings. Render automatically supplies `PORT` and `RENDER_EXTERNAL_URL`; the bot detects `RENDER_EXTERNAL_URL` and starts webhook mode. Set `WEBHOOK_SECRET` to a long random value. Render's public service URL should be HTTPS, which is required for Telegram webhooks.

A typical deployment sequence is:

```
Plain Text

Build command: pip install -r requirements.txt
Start command: python bot.py
Health check path: leave unset unless your Render plan is configured with a sep
```

After deployment, inspect the Render logs for `Starting webhook server`. If a previous local polling process is still running, stop it before testing webhook mode. Telegram delivers updates to exactly one active webhook or polling consumer for a bot token.

Environment variables

Variable	Required	Description
`BOT_TOKEN`	Yes	BotFather token
`OMDB_API_KEY`	Yes	OMDb movie metadata API key
`OPENAI_API_KEY`	Yes	OpenAI API key
`ADMIN_IDS`	Yes	Comma-separated numeric Telegram user IDs
`MAIN_CHANNEL_ID`	Yes	Main publication channel
`CLUB_CHANNEL_ID`	Yes	Club review channel
`DRIVE_CHANNEL_ID`	Yes	Storage channel for copied videos
`DIGITAL_SHOP_LINK`	No	Link inserted into both post formats
`OPENAI_MODEL`	No	Defaults to `gpt-4o-mini`

`OPENAI_API_BASE`	No	Defaults to `https://api.openai.com/v1`
`WEBHOOK_SECRET`	Recommended	Telegram webhook secret token
`WEBHOOK_PATH`	No	Defaults to `telegram-webhook`
`LOG_LEVEL`	No	Defaults to `INFO`

Production notes

The current workflow stores an in-progress job in `context.user_data`, which is appropriate for a single admin workflow but is intentionally not a durable database. For a multi-admin queue, crash recovery, audit history, or multiple simultaneous jobs per admin, add a persistent store such as PostgreSQL or Redis and a job identifier.

OpenAI and OMDb calls have bounded HTTP timeouts. Telegram/API failures are logged with stack traces while the admin receives a concise error. The bot does not retry channel publication automatically, preventing accidental duplicate posts after uncertain network failures. If a Club post succeeds but the subsequent Main post fails, the admin should manually publish or delete the partial result after checking the logs.

Telegram message captions have a length limit. The implementation sends the final preview as a normal text message and uses the poster only as a separate confirmation image, while the final Club and Main captions remain concise enough for normal movie copy. Very long model output should be shortened in the prompt or by adding a length guard before publication.

License and content responsibility

This code is a technical automation template. The operator is responsible for obtaining rights to copy, store, and distribute videos and posters, and for complying with Telegram, OMDb, OpenAI, Render, and local legal requirements.